Ques 1 Student Result Management

```
import java.io.*;

class Student {

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

int rollNo;
String name;
int marks1, marks2, marks3;

void studentDetail() throws IOException {
System.out.println("Enter name: ");
name = br.readLine();

System.out.println("Enter Roll number: ");
rollNo = Integer.parseInt(br.readLine());

System.out.println("Enter marks of subject1: ");
marks1 = Integer.parseInt(br.readLine());

System.out.println("Enter marks of subject2: ");
marks2 = Integer.parseInt(br.readLine());

System.out.println("Enter marks of subject3: ");
marks3 = Integer.parseInt(br.readLine());
}

int totalMarks() {
int total = marks1 + marks2 + marks3;
return total;
```

```
}

double calcPercent() {
double percentage = (totalMarks() / 300.0) * 100;
return percentage;
}

String grade() {
double percentage = calcPercent();

if (percentage >= 90) {
return "A+";
} else if (percentage >= 80) {
return "A";
} else if (percentage >= 70) {
return "B";
} else if (percentage >= 60) {
return "C";
} else if (percentage >= 50) {
return "D";
} else {
return "F";
}
}

void display() {
System.out.println("Roll No: " + rollNo);
System.out.println("Name: " + name);
System.out.println("Total Marks : " + totalMarks());
System.out.println("Percentage  : " + calcPercent() + "%");
System.out.println("Grade       : " + grade());
```

```
}
}

class StudentGrade {
public static void main(String[] args) throws IOException {
Student s1 = new Student();
Student s2 = new Student();

System.out.println("Enter details of Student 1:");
s1.studentDetail();

System.out.println("Enter details of Student 2:");
s2.studentDetail();

System.out.println("Details of Student 1:");
s1.display();

System.out.println("Details of Student 2:");
s2.display();
}
}
```

Ques 2 Bank Account Using Constructors

```
class BankAccount {
int accountNumber;
String accountHolderName;
double balance;

BankAccount() {
accountNumber = 0;
accountHolderName = "Not Given";
balance = 0;
```

```
}

BankAccount(int accountNumber, String accountHolderName, double balance) {
this.accountNumber = accountNumber;
this.accountHolderName = accountHolderName;
this.balance = balance;
}

void deposit(double amount) {
if(amount > 0) {
balance = balance + amount;
System.out.println("Deposited: " + amount);
} else {
System.out.println("Invalid deposit amount");
}
}

void withdraw(double amount) {
if(amount > balance) {
System.out.println("Insufficient balance");
} else if(amount <= 0) {
System.out.println("Invalid withdrawal amount");
} else {
balance = balance - amount;
System.out.println("Withdrawn: " + amount);
}
}

void displayAccount() {
System.out.println("Account Details: ");
System.out.println("Account Number: " + accountNumber);
```

```
System.out.println("Account Holder: " + accountHolderName);
System.out.println("Balance: " + balance);
}
}

class BankDetails {
public static void main(String[] args) {

BankAccount acc1 = new BankAccount();

BankAccount acc2 = new BankAccount(101, "Rahul", 25000);

acc1.displayAccount();
System.out.println();

acc2.displayAccount();
System.out.println();

acc2.deposit(5000);
acc2.withdraw(3000);
System.out.println();

acc2.displayAccount();
}
}
```

Ques 3 Shape Calculator Using Abstract Class

```
abstract class Shape {

abstract void calculateArea();
abstract void calculatePerimeter();
}
```

```
class Circle extends Shape {
double radius;

Circle(double radius) {
this.radius = radius;
}

void calculateArea() {
double area = 3.14 * radius * radius;
System.out.println("Circle Area: " + area);
}

void calculatePerimeter() {
double perimeter = 2 * 3.14 * radius;
System.out.println("Circle Perimeter: " + perimeter);
}
}

class Rectangle extends Shape {
double length, breadth;

Rectangle(double length, double breadth) {
this.length = length;
this.breadth = breadth;
}

void calculateArea() {
double area = length * breadth;
System.out.println("Rectangle Area: " + area);
}
```

```
void calculatePerimeter() {
double perimeter = 2 * (length + breadth);
System.out.println("Rectangle Perimeter: " + perimeter);
}
}

class Triangle extends Shape {
double base, height, side1, side2;

Triangle(double base, double height, double side1, double side2) {
this.base = base;
this.height = height;
this.side1 = side1;
this.side2 = side2;
}

void calculateArea() {
double area = 0.5 * base * height;
System.out.println("Triangle Area: " + area);
}

void calculatePerimeter() {
double perimeter = side1 + side2 + base;
System.out.println("Triangle Perimeter: " + perimeter);
}
}

class ShapeCalculator {
public static void main(String[] args) {
```

```
Shape s[] = new Shape[3];

s[0] = new Circle(4);
s[1] = new Rectangle(5, 2);
s[2] = new Triangle(6, 5, 4, 4);

for(int i = 0; i < s.length; i++) {
s[i].calculateArea();
s[i].calculatePerimeter();
System.out.println();
}
}
}
```

Ques 4 Employee Salary System Using Abstract Class

```
import java.io.*;

abstract class Employee {
int employeeId;
String employeeName;
double basicSalary;

Employee(int employeeId, String employeeName, double basicSalary) {
this.employeeId = employeeId;
this.employeeName = employeeName;
this.basicSalary = basicSalary;
}

abstract double calculateSalary();

void displayEmployee() {
System.out.println("Employee ID: " + employeeId);
```

```
System.out.println("Employee Name: " + employeeName);
System.out.println("Basic Salary: " + basicSalary);
}
}

class PermanentEmployee extends Employee {

PermanentEmployee(int employeeId, String employeeName, double basicSalary) {
super(employeeId, employeeName, basicSalary);
}

double calculateSalary() {
double hra = 0.20 * basicSalary;
double da = 0.40 * basicSalary;
double pf = 0.12 * basicSalary;

double grossSalary = basicSalary + hra + da;
double netSalary = grossSalary - pf;

return netSalary;
}
}

class ContractEmployee extends Employee {

ContractEmployee(int employeeId, String employeeName, double basicSalary) {
super(employeeId, employeeName, basicSalary);
}

double calculateSalary() {
double allowance = 0.10 * basicSalary;
```

```
double grossSalary = basicSalary + allowance;

return grossSalary;
}
}

class SalarySystem {
public static void main(String[] args) throws IOException {
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

Employee e;

System.out.println("Enter Permanent Employee Details:");

System.out.print("Enter Employee ID: ");
int e1 = Integer.parseInt(br.readLine());

System.out.print("Enter Employee Name: ");
String name1 = br.readLine();

System.out.print("Enter Basic Salary: ");
double salary1 = Double.parseDouble(br.readLine());

e = new PermanentEmployee(e1, name1, salary1);

System.out.println("\n     Permanent Employee");
e.displayEmployee();
System.out.println("Net Salary: " + e.calculateSalary());

System.out.println("\nEnter Contract Employee Details:");
```

```
System.out.print("Enter Employee ID: ");
int e2 = Integer.parseInt(br.readLine());

System.out.print("Enter Employee Name: ");
String name2 = br.readLine();

System.out.print("Enter Basic Salary: ");
double salary2 = Double.parseDouble(br.readLine());

e = new ContractEmployee(e2, name2, salary2);

System.out.println("\n      Contract Employee");
e.displayEmployee();
System.out.println("Gross Salary: " + e.calculateSalary());
}
}
```

Ques 5 Payment System Using Interface

```
import java.io.*;

interface Payment {
void makePayment(double amount);
void paymentDetails();
}

class CreditCardPayment implements Payment {
String cardNumber;
String cardHolderName;
double amount;

CreditCardPayment(String cardNumber, String cardHolderName) {
this.cardNumber = cardNumber;
```

```
this.cardHolderName = cardHolderName;
}

public void makePayment(double amount) {
if(amount <= 0) {
System.out.println("Invalid payment amount.");
} else {
this.amount = amount;
System.out.println("\nPayment Successful!");
}
}

public void paymentDetails() {
System.out.println("\nPayment Mode: Credit Card");
System.out.println("Card Number: " + cardNumber);
System.out.println("Card Holder: " + cardHolderName);
System.out.println("Amount: Rs. " + amount);
}
}

class UPIPayment implements Payment {
String upiId;
String userName;
double amount;

UPIPayment(String upiId, String userName) {
this.upiId = upiId;
this.userName = userName;
}

public void makePayment(double amount) {
```

```
if(amount <= 0) {
System.out.println("Invalid payment amount.");
} else {
this.amount = amount;
System.out.println("\nPayment Successful!");
}
}

public void paymentDetails() {
System.out.println("\nPayment Mode: UPI");
System.out.println("UPI ID: " + upiId);
System.out.println("User Name: " + userName);
System.out.println("Amount: Rs. " + amount);
}
}

class CashPayment implements Payment {
String customerName;
double amount;

CashPayment(String customerName) {
this.customerName = customerName;
}

public void makePayment(double amount) {
if(amount <= 0) {
System.out.println("Invalid payment amount.");
} else {
this.amount = amount;
System.out.println("\nPayment Successful!");
}
```

```
}

public void paymentDetails() {
System.out.println("\nPayment Mode: Cash");
System.out.println("Customer Name: " + customerName);
System.out.println("Amount: Rs. " + amount);
}
}

class PaymentSystem {
public static void main(String[] args) throws IOException {

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

System.out.println("      PAYMENT SYSTEM");
System.out.println("1. Credit Card");
System.out.println("2. UPI");
System.out.println("3. Cash");

System.out.print("Enter Choice: ");
int choice = Integer.parseInt(br.readLine());

Payment p;

switch(choice) {

case 1:
System.out.print("Enter Card Number: ");
String cardNumber = br.readLine();

System.out.print("Enter Card Holder Name: ");
```

```
String cardHolderName = br.readLine();

System.out.print("Enter Amount: ");
double cardAmount = Double.parseDouble(br.readLine());

p = new CreditCardPayment(cardNumber, cardHolderName);
p.makePayment(cardAmount);
p.paymentDetails();

break;

case 2:
System.out.print("Enter UPI ID: ");
String upiId = br.readLine();

System.out.print("Enter User Name: ");
String userName = br.readLine();

System.out.print("Enter Amount: ");
double upiAmount = Double.parseDouble(br.readLine());

p = new UPIPayment(upiId, userName);
p.makePayment(upiAmount);
p.paymentDetails();

break;

case 3:
System.out.print("Enter Customer Name: ");
String customerName = br.readLine();
```

```
System.out.print("Enter Amount: ");
double cashAmount = Double.parseDouble(br.readLine());

p = new CashPayment(customerName);
p.makePayment(cashAmount);
p.paymentDetails();

break;

default:
System.out.println("Invalid choice.");
}
}
}
```